



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET	V2.0.6	DOWNLOADS	1.1K	LICENSE	MIT	GITHUB PAGES	ONLINE
-------	--------	-----------	------	---------	-----	--------------	--------

Analisi del codice GridUtility

1. Scopo generale

Il file `GridUtility.vb` contiene una classe **utility** (`GridUtility`) dedicata alla configurazione e manipolazione di oggetti `DataGridView` in un'applicazione Windows Forms.

La classe è **non ereditabile** (`NotInheritable`) e fornisce due metodi pubblici statici:

1. `Initialize(dgv As DataGridView)` – imposta le proprietà di stile e comportamento di un `DataGridView` in modo coerente con il tema dell'applicazione.
2. `ConvertiMaiuscolo(dgv As DataGridView)` – converte tutti i valori di tipo stringa presenti nelle celle in maiuscolo.

Questa struttura permette di centralizzare la logica di formattazione, riducendo la duplicazione di codice in più form o controlli.

2. Componenti chiave e funzioni

Elemento	Descrizione	Note
Costruttore privato	<code>Private Sub New()</code>	Previene l'istanza della classe; tutti i metodi sono statici.
Font statici	<code>FontBold</code> , <code>FontRegular</code>	Definiscono due font predefiniti (bold e regular) da usare nei <code>DataGridView</code> .
<code>Initialize</code>	Configura le proprietà di base, l'aspetto dell'header, le righe, le righe alternate, l'altezza delle righe e i colori di selezione.	Utilizza classi di supporto (<code>Colori</code> , <code>ThemeFonts</code> , <code>FrameworkLogger</code>) che non sono incluse nel snippet.

Elemento	Descrizione	Note
<code>ConvertiMaiuscolo</code>	Scorre tutte le righe e le celle, trasformando i valori stringa in maiuscolo.	Gestisce eccezioni e logga eventuali errori.
<code>FrameworkLogger</code>	Classe di logging (presumibilmente).	Non è definita qui, ma è usata per registrare errori.
<code>Colori</code>	Classe di utilità per ottenere colori definiti dal tema.	Metodo <code>GetColor</code> restituisce un <code>Color</code> .
<code>ThemeFonts</code>	Classe di utilità per ottenere font predefiniti.	Metodo <code>DefaultFontBold</code> restituisce un <code>Font</code> .

3. Flusso di esecuzione

3.1 Initialize

1. Controllo di nullità

```
1 If dgv Is Nothing Then Exit Sub
```

Se il parametro è `Nothing`, la routine termina subito.

2. Configurazione di base

- Disabilita lo stile visuale degli header.
- Imposta il bordo a `None`.
- Disabilita l'aggiunta/eliminazione di righe da parte dell'utente.
- Imposta la selezione a singola riga (`FullRowSelect`).
- Imposta l'auto-ridimensionamento delle colonne a `Fill`.
- Nasconde le intestazioni di riga e rende il controllo **read-only**.
- Imposta colori di sfondo e griglia tramite `Colori.GetColor`.

3. Stile dell'header

- Colore di sfondo e testo.
- Font in grassetto (`ThemeFonts.DefaultFontBold`).
- Allineamento centrato.
- Altezza header a 35 pixel.

4. Stile delle righe

- Font di default (`ThemeFonts.DefaultFont`).
- Colori di testo, sfondo e selezione.
- Colore di riga alternata a `WhiteSmoke`.
- Altezza riga a 30 pixel.

5. Rimozione del colore di selezione "blu Windows"

- Imposta nuovamente `SelectionBackColor` e `SelectionForeColor` per sovrascrivere il colore di default di Windows.

6. Gestione eccezioni

- Tutto il blocco è avvolto in `Try/Catch`.
- In caso di errore, viene chiamato `FrameworkLogger.LogError`.

3.2 ConvertiMaiuscolo

1. Controlli preliminari

- Se il `DataGridView` è `Nothing` o non contiene righe, la routine termina.

2. Iterazione su righe e celle

```

1  For Each row As DataGridViewRow In dgv.Rows
2      For Each cell As DataGridViewCell In row.Cells
3          If TypeOf cell.Value Is String Then
4              cell.Value = cell.Value.ToString().ToUpper()
5          End If
6      Next
7  Next

```

- Per ogni cella, se il valore è una stringa, viene convertito in maiuscolo con `ToUpper()`.

3. Gestione eccezioni

- Come `Initialize`, l'intero ciclo è protetto da `Try/Catch` e logga eventuali errori.

4. Possibili miglioramenti / errori

Problema	Suggerimento
Duplicazione di controlli <code>If dgv Is Nothing</code>	Rimuovere il secondo controllo all'interno di <code>ConvertiMaiuscolo</code> .
Uso di <code>Colori.GetColor</code> e <code>ThemeFonts</code>	Assicurarsi che queste classi siano disponibili e gestiscano correttamente i casi di colore non trovato.
Gestione delle eccezioni	Potrebbe essere utile propagare l'eccezione dopo il log, se il chiamante deve reagire.
Performance	Per <code>DataGridView</code> molto grandi, la conversione in maiuscolo può essere costosa; considerare l'uso di <code>BeginUpdate/EndUpdate</code> o di un flag per disabilitare il rendering temporaneamente.
Stile alternato	L'uso di <code>Color.WhiteSmoke</code> è hard-coded; potrebbe essere sostituito da un colore del tema.

5. Come usarlo

```
1 ' Inizializza un DataGridView con lo stile predefinito
2 GridUtility.Initialize(Me.ClientiDataGridView)
3
4 ' Converte tutti i valori stringa in maiuscolo
5 GridUtility.ConvertiMaiuscolo(Me.ClientiDataGridView)
```

Questa utility rende più semplice mantenere un aspetto coerente tra tutti i DataGridView dell'applicazione e fornisce una funzione di utilità comune (convertire in maiuscolo).

Spiegazione del Codice Evidenzia Test

1. Scopo Generale

Il metodo `EvidenziaTesto` è una procedura condivisa (static) che permette di evidenziare in un `DataGridView` tutte le celle che contengono una determinata stringa di testo. L'evidenziazione avviene cambiando il colore di sfondo delle celle che soddisfano il criterio di ricerca.

2. Componenti Chiave

Elemento	Descrizione	Note
<code>dgv As DataGridView</code>	Il controllo <code>DataGridView</code> su cui operare.	Deve essere già inizializzato e popolato.
<code>testo As String</code>	Testo da cercare all'interno delle celle.	La ricerca è case-insensitive.
<code>Optional colore As Integer = 5</code>	Indice di colore (default 5).	Viene passato a <code>Colori.GetColor</code> per ottenere il colore effettivo.
<code>Try...Catch</code>	Blocco di gestione eccezioni.	In caso di errore, viene registrato tramite <code>FrameworkLogger.LogError</code> .
<code>Colori.GetColor(colore)</code>	Funzione (presumibilmente) che restituisce un oggetto <code>Color</code> a partire dall'indice.	Non è mostrata nel frammento, ma è responsabile della conversione.
<code>FrameworkLogger.LogError</code>	Logger personalizzato.	Serve per tracciare errori senza interrompere l'applicazione.

3. Flusso di Esecuzione

1. Validazione di input

- Se `dgv` è `Nothing` o non contiene righe (`Rows.Count = 0`), la procedura termina subito.
- Se `testo` è `Nothing`, vuoto o contiene solo spazi, la procedura termina.

2. Iterazione sulle righe e sulle celle

- Per ogni riga (`DataGridViewRow`) del `DataGridView`.
- Per ogni cella (`DataGridViewCell`) della riga corrente.

3. Controllo del valore della cella

- Se la cella ha un valore (`cell.Value IsNot Nothing`) e quel valore, convertito in stringa, contiene la stringa `testo` (ricerca case-insensitive grazie a `StringComparison.OrdinalIgnoreCase`), allora:
 - Viene impostato il colore di sfondo della cella con `cell.Style.BackColor = Colori.GetColor(colore)`.

4. Gestione eccezioni

- Qualsiasi eccezione sollevata durante l'esecuzione viene catturata e loggata con `FrameworkLogger.LogError`.
-

4. Possibili Miglioramenti e Errori

Problema	Descrizione	Suggerimento
Performance su grandi dataset	L'algoritmo è $O(n*m)$ (righe $\times$ celle) e può diventare lento con tabelle molto grandi.	Utilizzare <code>DataGridView</code> con modalità virtuale o filtrare prima le righe interessate.
Uso di <code>IndexOf</code> su <code>ToString()</code>	Se la cella contiene oggetti non stringa, la conversione può essere costosa o generare risultati inattesi.	Controllare il tipo di dato o usare <code>cell.Value?.ToString()</code> con gestione di <code>null</code> .
Colore hard-coded	Il valore predefinito <code>5</code> è arbitrario e dipende dall'implementazione di <code>Colori.GetColor</code> .	Fornire un enum o costanti per i colori, rendendo il codice più leggibile.
Logging senza contesto	Il messaggio di log è generico.	Includere informazioni come il nome della colonna o l'indice della riga per facilitare il debug.
Non reset del colore	Se la procedura viene chiamata più volte con testi diversi, le celle	Aggiungere un passaggio per ripristinare il colore di sfondo

Problema	Descrizione	Suggerimento
	precedentemente evidenziate rimangono colorate.	originale prima di applicare nuove evidenziazioni.

5. Esempio di Utilizzo

```

1 ' Evidenzia tutte le celle che contengono la stringa "RAM" nel
   DataGridView DgvOrdini
2 GridUtility.EvidenziaTesto(DgvOrdini, "RAM")

```

In questo esempio, tutte le celle che contengono “RAM” (in qualsiasi combinazione di maiuscole/minuscole) verranno colorate con il colore associato all’indice 5.

Conclusione

Il metodo `EvidenziaTesto` è una utility semplice e utile per evidenziare rapidamente contenuti specifici in un `DataGridView`. Con alcune ottimizzazioni e miglioramenti di robustezza, può diventare una componente affidabile di un’applicazione Windows Forms.

Analisi del codice Filtra Tutti

1. Scopo generale

Il file contiene una classe di utilità (`GridUtility` – non mostrata ma implicita) che fornisce metodi statici per:

1. **Filtrare** i dati visualizzati in un `DataGridView` in base a un testo di ricerca.
2. **Colorare** lo sfondo di celle che contengono valori null o vuoti.
3. **Estrarre** il `DataTable` originale da un `DataGridView` indipendentemente dal tipo di `DataSource` (`DataTable`, `DataGridView`, `BindingSource`).

Queste funzioni sono pensate per semplificare le operazioni comuni di interfaccia utente in applicazioni WinForms che mostrano dati tabellari.

2. Componenti principali

Metodo	Parametri	Funzione
<code>FiltraTutti</code>	<code>dgv As DataGridView</code> , <code>dtOriginal As DataTable</code> ,	Filtra tutti i campi di <code>dtOriginal</code> con la stringa <code>testo</code> e imposta il

Metodo	Parametri	Funzione
	testo As String	risultato come DataSource del DataGridView .
FiltraTesto	dgv As DataGridView , testo As String	Recupera il DataTable associato al DataGridView (indipendentemente dal tipo di DataSource) e chiama FiltraTutti .
ImpostaColNull	dgv As DataGridView , nomeColonna As String , backColor As Color = Nothing , foreColor As Color = Nothing	Evidenzia le celle della colonna indicata che contengono valori null o vuoti, usando colori di default (giallo/azzurro) se non specificati.

Dettagli di implementazione

- **Sicurezza:** tutti i metodi verificano che l'oggetto dgv non sia Nothing e che le colonne esistano prima di procedere.
- **Gestione degli errori:** FiltraTutti è avvolto in un blocco Try/Catch che registra l'eccezione tramite FrameworkLogger.LogError .
- **Costruzione del filtro:** per ogni colonna di dtOriginal si crea una clausola LIKE (per stringhe) o una conversione a stringa (per altri tipi) e si concatenano con OR .

```
1 filtro &= $"[{col.ColumnName}] LIKE '%{testo.Replace("'", "'')}%'"
```

- **DataView:** viene creato un nuovo DataView con RowFilter impostato al filtro costruito, e questo DataView diventa la nuova fonte dati del DataGridView .
- **Colorazione:** il metodo scorre tutte le righe (escluse le righe di inserimento) e controlla se il valore è Nothing , DBNull o una stringa vuota. Se sì, imposta i colori di sfondo e testo; altrimenti ripristina i colori di default del DataGridView .

3. Flusso di esecuzione

3.1 FiltraTutti

1. **Validazione:** se dgv o dtOriginal sono Nothing , il metodo termina subito.
2. **Ricerca vuota:** se testo è vuoto o solo spazi, il DataSource viene riportato a dtOriginal (tutti i record) e il metodo termina.
3. **Costruzione del filtro:**
 - Inizializza filtro come stringa vuota.
 - Per ogni colonna di dtOriginal :
 - Se non è la prima colonna, aggiunge " OR " .

- Se la colonna è di tipo `String`, aggiunge una clausola `LIKE` con escape per l'apostrofo.
 - Altrimenti, converte la colonna a stringa e applica `LIKE`.
4. **Applicazione del filtro:** crea un `DataRowView` su `dtOriginal`, imposta `RowFilter` al filtro costruito e assegna il `DataRowView` al `DataSource` del `DataGridView`.
 5. **Gestione eccezioni:** eventuali errori vengono registrati.

3.2 FiltraTesto

1. **Recupero del DataTable:** controlla il tipo di `DataSource` del `DataGridView` (`DataTable`, `DataRowView`, `BindingSource`) e ottiene il `DataTable` corrispondente.
2. **Chiamata di FiltraTutti:** se il `DataTable` è stato trovato, chiama `FiltraTutti(dgv, dt, testo)` (commentato come "Versione veloce").
3. **Fine:** se non è stato trovato un `DataTable`, il metodo termina senza fare nulla.

3.3 ImpostaColNull

1. **Validazione:** verifica che `dgv` non sia `Nothing` e che la colonna esista.
2. **Colori di default:** se `BackColor` o `ForeColor` non sono specificati, li impostano rispettivamente a `Yellow` e `Blue`.
3. **Iterazione delle righe:** per ogni riga non nuova:
 - Recupera il valore della cella nella colonna indicata.
 - Determina se è null/DBNull/empty.
 - Se è null, imposta i colori di sfondo e testo; altrimenti ripristina i colori di default del `DataGridView`.

4. Possibili miglioramenti e punti di attenzione

Problema	Suggerimento
Escaping del testo	L'escape di apostrofi è corretto, ma per sicurezza si potrebbe usare <code>SqlParameter</code> o un metodo di escape più generico.
Performance	Per tabelle molto grandi, costruire una stringa di filtro lunga può essere costoso. Una soluzione alternativa è filtrare direttamente sul <code>DataRowView</code> con una query più efficiente.
Tipo di dato	La conversione <code>Convert([col], 'System.String')</code> funziona per la maggior parte dei tipi, ma potrebbe generare errori per tipi complessi (es. <code>DateTime</code>).
Gestione di BindingSource	Se il <code>BindingSource</code> è collegato a un oggetto diverso da <code>DataTable</code> / <code>DataRowView</code> , il metodo non gestisce quel caso.
Colori di default	L'uso di <code>Nothing</code> per confrontare <code>Color</code> è valido in VB.NET, ma può essere più chiaro usare <code>Color.Empty</code> .

Problema	Suggerimento
Documentazione	Aggiungere XML comments ai metodi migliorerebbe la leggibilità e l'autocompletamento in IDE.

5. Conclusione

Il codice fornisce un set di strumenti utili per la gestione di `DataGridView` in WinForms:

Note

- **Filtraggio dinamico** di tutti i campi con una singola stringa di ricerca.
- **Estrazione** del `DataTable` originale da varie tipologie di `DataSource`.
- **Evidenziazione** visiva di celle con valori null o vuoti.

Con alcune piccole ottimizzazioni e una documentazione più completa, questi metodi possono diventare parte integrante di una libreria di utilità UI.